

## Revision of Date, Time and TimeStamp in JDBC

13 September 2026 08:52

**java.sql.Date:** The `java.sql.Date` extends `java.util.Date` class. The `java.sql.Date` is used in the JDBC API, as a wrapper around `java.util.Date` that handles SQL specific requirements. The class exists to represent SQL date which keeps year, month and days with no time. **java.sql.Date** should be used only when working with databases, like to set a date on a `java.sql.PreparedStatement`. Get a date from a `java.sql.ResultSet`, Working with `java.sql.SQLDate` etc.

We can instantiate a `java.sql.Date` object using the following constructor:

- `Date(long date)`: Constructs a date object using the given milliseconds time value. For example:  
WAP to create a `java.sql.Date` object and print the time in milliseconds and the `java.sql.Date` object.

```
public class DateandTime {
    public static void main(String[] args) {
        //java.util.Date date = new java.util.Date();
        long now = System.currentTimeMillis();
        Date date = new Date(now);
        System.out.println("Time in milliseconds: "+now);
        System.out.println("sql date: "+date);

        System.out.println("time in milliseconds using sql.Date object:
"+date.getTime());
    }
}
```

*java.util.Date*  
↑  
*java.sql.Date*

Output:

```
Time in milliseconds: 1789272168095
sql date: 2026-09-13
time in milliseconds using sql.Date object: 1789272168095
```

Or, you can use static `valueOf(...)` method to get an instance of `java.sql.Date` object.

**static Date valueOf(String s):** Convert the string in JDBC date escape format to a date value.

**static Date valueOf(LocalDate date):** Obtains an instance of `Date` from a local `Date` object with the same year, month, and day of the month value as given **LocalDate**.

For example: Use the above methods to print the Date.

```
package MySQLPrograms;

import java.sql.Date;
import java.time.LocalDate;
import java.time.Month;
```

```

public class SQLDateValueOfExample {
    public static void main(String[] args) {
        Date sqldate1 = Date.valueOf("1983-11-14");
        System.out.println("sql Date 1: "+sqldate1);

        Date sqldate2 = Date.valueOf(LocalDate.of(2004, Month.NOVEMBER, 30));
        System.out.println("sql Date 2: "+sqldate2);
    }
}

```

#### Output:

```

sql Date 1: 1983-11-14
sql Date 2: 2004-11-30

```

#### java.sql.Date Methods

- void setTime(long time): Set an existing Date object using the given millisecond time value.
- LocalDate toLocalDate(): Creates a local date instance using the year, month and date from this Date object.
- String toString(): Format the date in the Date escape format "YYYY-MM-DD".

**java.time.Instant**

An instantaneous point on the time-line.

This class models a single instantaneous point on the time-line. This might be used to record event time-stamps in the application.

The range of an instant requires the storage of a number larger than a **long**. To achieve this, the class stores a **long** representing epoch-seconds and an **int** representing nanosecond-of-second, which will always be between 0 and 999,999,999. The epoch-seconds are measured from the standard Java epoch of **1970-01-01 00:00:00Z** where instants after the epoch have positive values, and earlier instants have negative values. For both the epoch-second and nanosecond parts, a larger value is always later on the time-line than a smaller value.

|                                |                                    |
|--------------------------------|------------------------------------|
| InstantiationError             | java.lang                          |
| InstantiationException         | java.lang                          |
| Instant                        | java.time                          |
| InstantSource                  | java.time                          |
| Instantiation                  | java.security.DrbgParameters       |
| Instrumentation                | java.lang.instrument               |
| InstalledProviders             | oracle.jdbc.driver.resource        |
| InstanceAlreadyExistsException | javax.management                   |
| InstanceNotFoundException      | javax.management                   |
| InstanceProperty               | oracle.jdbc.internal.OracleConn... |
| InstantValueEncoder            | com.mysql.cj.protocol.a            |
| Instrument                     | javax.sound.midi                   |

← not present in java.sql.Date

```

package MyJDBCOne;
import java.sql.Date;
import java.time.Instant;
import java.time.LocalDate;
public class SQLDateMethods {
    public static void main(String[] args) {
        long now = System.currentTimeMillis();
        Date sqlDate = new Date(now);
        System.out.println("SQLDate: " + sqlDate);
        sqlDate.setTime(356373000000L);
        System.out.println("sqlDate: " + sqlDate);
        LocalDate localDt = sqlDate.toLocalDate();
        System.out.println("LocalDate: " + localDt);
    }
}

```

```

        Instant instant = sqlDate.toInstant();
        System.out.println("Instant: " + instant);
    }
}

```

#### Output:

```

SQLDate:2026-09-13
sqlDate: 1981-04-17
LocalDate: 1981-04-17
Exception in thread "main" java.lang.UnsupportedOperationException
    at java.sql/java.sql.Date.toInstant(Date.java:316)
    at MyJDBCOne.SQLDateMethods.main(SQLDateMethods.java:17)

```

*Instant cannot be used with  
java.sql.Date.*

*java.util.Date  
↑  
java.sql.Timestamp.*

#### java.sql.Timestamp

The **java.sql.Timestamp** extends **java.util.Date** class. **java.sql.Timestamp** is used in the JDBC API, as a wrapper around **java.util.Date** that handles SQL specific requirements. This class exists to represent SQL timestamp which able to keep the date and time. Similar to **java.sql.Date** and **java.sql.Time**, **java.sql.Timestamp** should be used only when working with databases, like to set a timestamp on **java.sql.PreparedStatement**, get a timestamp from a **java.sql.ResultSet**, working with **java.sql.SQLData** etc.

#### Getting java.sql.Timestamp Instance

We can instantiate a **java.sql.Timestamp** object using the following constructor:

- **Timestamp(long time)**: Constructs a Timestamp object using a millisecond time value.

- And the **valueOf(String str)** method can be used to create Timestamp object:

**public static [Timestamp](#) valueOf([String](#) s)**

Converts a String object in JDBC timestamp escape format to a Timestamp value.

Parameters:

s - timestamp in format yyyy-[m]m-[d]d hh:mm:ss[.f...]. The fractional seconds may be omitted. The leading zero for mm and dd may also be omitted.

Returns:

corresponding Timestamp value

Throws:

[IllegalArgumentException](#) - if the given argument does not have the format yyyy-[m]m-[d]d hh:mm:ss[.f...]

- **public static [Timestamp](#) valueOf([LocalDateTime](#) dateTime)**

Obtains an instance of Timestamp from a **LocalDateTime** object, with the same year, month, day of month, hours, minutes, seconds and nanos date-time value as the provided **LocalDateTime**.

The provided **LocalDateTime** is interpreted as the local date-time in the local time zone.

Parameters:

dateTime - a **LocalDateTime** to convert

Returns:

a Timestamp object

Throws:

[NullPointerException](#) - if dateTime is null.

Since:

1.8

From <[<https://docs.oracle.com/en/java/javase/25/docs/api/java.sql/java.sql/Timestamp.html#valueOf\(java.lang.String\)>](https://docs.oracle.com/en/java/javase/25/docs/api/java.sql/java.sql/Timestamp.html#valueOf(java.lang.String))>

WAP to create three Timestamp object, one by using constructor, another by using valueOf(String s) method, and the last one using valueOf(LocalDateTime dateTime) metod, and print the information stored in all three objects.

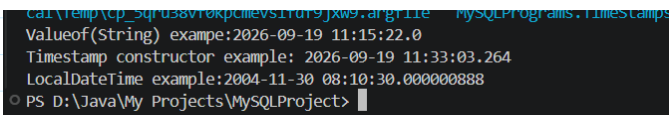
```
import java.sql.Timestamp;
import java.time.LocalDateTime;

public class TimeStamps {
    public static void main(String[] args) {
        Timestamp obj1= Timestamp.valueOf("2026-09-19 11:15:22");
        //Value of is a factory method - A static method that returns the object of its class is
        //called factory method.
        System.out.println("Valueof(String) exampe:"+obj1);

        long now=System.currentTimeMillis();
        Timestamp obj2= new Timestamp(now);
        System.out.println("Timestamp constructor example: "+obj2);

        Timestamp obj3=Timestamp.valueOf(LocalDateTime.of(2004, 11, 30, 8, 10, 30, 888));
        System.out.println("LocalDateTIme example:"+obj3);
    }
}
```

Output:



```
CarTemp(cp_SqlDb387rkpcmevstf0t9jxw9,grt11e MySQLPrograms,TimeStamps
Valueof(String) exampe:2026-09-19 11:15:22.0
Timestamp constructor example: 2026-09-19 11:33:03.264
LocalDateTIme example:2004-11-30 08:10:30.000000888
PS D:\Java\My Projects\MySQLProject>
```

Another method is from(...) used to create Timestamp object:

- static Timestamp from(Instant instant): obtains an instance of Timestamp from an **Instant object**.

WAP to create a Timestamp object using Instant class.

```
package MySQLPrograms;

import java.sql.Timestamp;
import java.time.Instant;

public class SQLTimestampFromTOINSTANT {
    public static void main(String[] args) {
        //Instant object to Timestamp
        Instant instant1=Instant.now();
        Timestamp obj=Timestamp.from(instant1);

        System.out.println("Timetamp from(Instance instance) example:"+obj);
    }
}
```

```

//Timestamp object to Instant
Instant instant2 = obj.toInstant();
System.out.println("Instant 2:"+instant2);

    }
}

```

Output:

```

PS D:\Java\My Projects\MySQLProject> & C:\Program Files\jdk-24.0.1\bin\java.exe MySQLPrograms.SQLTimestampFromTOINSTANT
Timestamp from(Instance instance) example:2026-09-19 11:49:10.924102
Instant 2:2026-09-19T06:19:10.924102Z
PS D:\Java\My Projects\MySQLProject>

```

### Timestamp's Nanoseconds

One advantage in the java.sql.Timestamp from its super class java.util.Date is its ability to hold the nanosecond(which Date is only able to hold until milliseconds only). You can get and set the nanosecond using getNanos() and setNanos().

WAP to Create a Timestamp object, print the nano second using getNanos() method, set nano second using setNanos(), and print the value of nano second again.

```

package MySQLPrograms;

import java.sql.Date;
import java.sql.Timestamp;
import java.text.DateFormat;
import java.text.SimpleDateFormat;

public class GetNanosSetNanos {
    public static void main(String[] args) {
        DateFormat df=new SimpleDateFormat("YYYY-MM-dd HH:mm:ss:SSS");
        long now=System.currentTimeMillis();
        Date date = new Date(now);
        System.out.println("Date: "+df.format(date));

        Timestamp ts=new Timestamp(now);
        System.out.println("Timestamp :"+ts);

        System.out.println("timestamp nanosecond: "+ts.getNanos());
        ts.setNanos(123456789);
        System.out.println("New Timestamp in nanoseconds:"+ts.getNanos());
        System.out.println("sql new timestamp:"+ts);

    }
}

```

Output:

```
PS D:\Java\My Projects\MySQLProject> cd "D:\Java\My Projects\MySQLProject";  
cal\Temp\cp_5gru38vf0kpcmevsifdf9jxw9.argfile" 'MySQLPrograms.GetNanosSetNanos'  
Date: 2026-09-19 12:30:13.484  
Timestamp :2026-09-19 12:30:13.484  
timestamp nanosecond: 484000000  
New Timestamp in nanoseconds:123456789  
sql new timestamp:2026-09-19 12:30:13.123456789  
PS D:\Java\My Projects\MySQLProject>
```